OOP Python & Java Interview Questions with Answers

Q: What is OOP?
Answer: OOP is a programming paradigm that organizes software using objects containing data and methods.

Q: What are the four pillars of OOP?
Answer: Encapsulation, Inheritance, Polymorphism, and Abstraction.

Q: Why do we use OOP?
Answer: To improve code reusability, modularity, scalability, and maintainability.

Q: Difference between procedural and OOP?
Answer: Procedural focuses on functions; OOP focuses on objects and data.

Q: Difference between class and object?
Answer: Class is a blueprint; object is an instance of a class.

Q: What is self?
Answer: Reference to the current object; used to access instance members.

Q: Difference between self and cls?
Answer: self refers to an object; cls refers to the class.

Q: What is __init__()?
Answer: Constructor called automatically when an object is created.

Q: What if __init__() is not defined?
Answer: Python provides a default constructor.

Q: What is __new__()?
Answer: Creates and returns a new instance before __init__ initializes it.

Q: Instance vs class variables?
Answer: Instance variables belong to objects; class variables are shared by all objects.

Q: Instance/class/static methods?
Answer: Instance methods use self, class methods use cls and @classmethod, static methods use @staticmethod and neither self nor cls.

Q: Duck typing?
Answer: If an object behaves correctly, its type is not important.

Q: MRO?
Answer: Method Resolution Order determines the order of searching methods in inheritance.

Q: Diamond problem?
Answer: Multiple inheritance ambiguity resolved by Python's MRO.

Q: super()?
Answer: Calls parent class methods or constructor.

Q: Multiple inheritance?
Answer: A class inherits from more than one parent.

Q: _var vs __var vs __var__?
Answer: _var=protected convention, __var=name mangling, __var__=special magic methods.

Q: Is Python truly OOP?
Answer: Yes, almost everything is an object.

Q: Why isn't Java completely pure OOP?
Answer: Because it has primitive data types like int, char, boolean.

Q: Abstract class vs interface?
Answer: Abstract class can have implemented methods; interface defines a contract (modern Java also allows default methods).

Q: Constructor overloading?
Answer: Multiple constructors with different parameter lists.

Q: Overloading vs overriding?
Answer: Overloading: same name, different parameters. Overriding: child redefines parent method.

Q: Why no multiple inheritance in Java classes?
Answer: To avoid the diamond problem; interfaces are used instead.

Q: final keyword?
Answer: Prevents inheritance, overriding, or reassignment depending on context.

Q: this keyword?
Answer: Refers to the current object.

Q: Access modifiers?
Answer: public, private, protected, and default.

Q: Why getters/setters?
Answer: To validate and control access to private data.

Q: Can encapsulation exist without inheritance?
Answer: Yes. They are independent OOP concepts.

Q: Why encapsulation?
Answer: Data hiding, validation, and maintainability.

Q: private in Java vs __ in Python?
Answer: Java private is compiler-enforced; Python __ uses name mangling.

Q: Compile-time polymorphism?
Answer: Method overloading.

Q: Runtime polymorphism?
Answer: Method overriding.

Q: Does Python support method overloading?
Answer: Not directly; simulate using default arguments or *args.

Q: Operator overloading?
Answer: Redefining operators using special methods like __add__.

Q: What is abstraction?
Answer: Hiding implementation and exposing only essential functionality.

Q: Why abstract classes?
Answer: To define common behavior and enforce implementation.

Q: Can abstract class be instantiated?
Answer: No.

Q: Abstraction vs encapsulation?
Answer: Abstraction hides implementation; encapsulation hides data.

Coding Questions

1. Create a Student class with __init__() constructor.
2. Demonstrate inheritance using Animal->Dog.

3. Override a parent method in child class.
4. Show encapsulation using private fields/getters-setters in Java and __ variables in Python.
5. Create an abstract class using abc.ABC and implement it.